

# BFSI Personal Loan Voice Agent Report

Abrar Riyaz

May 1, 2026

## GitHub Repository:

[https://github.com/AbrarAhmed435/BFSIpersonalLoan\\_agent](https://github.com/AbrarAhmed435/BFSIpersonalLoan_agent)

## 1 Overview

This project implements a voice-based AI calling system for personal loan qualification. The backend triggers outbound calls using Vapi, and after the call ends, the transcript is processed to extract structured information such as user intent and financial details.

## 2 Tech Stack

- **Backend:** Flask (Python)
- **Voice Platform:** Vapi API
- **Telephony:** Phone call via Vapi (Twilio-backed)
- **NLU:** Custom rule-based intent classification and slot extraction
- **Configuration:** Environment variables (`.env`)
- **Tunneling:** ngrok (for webhook exposure)
- **Storage:** JSON file (`calls.json`)

## 3 System Flow

1. A call is triggered using CLI or API endpoint.
2. Backend sends a POST request to Vapi (`/call/phone`).
3. Vapi initiates the call and handles the conversation.
4. After the call ends, Vapi sends a POST request to the webhook endpoint.
5. Backend receives transcript and runs NLU (`analyze()`).
6. Extracted data is stored in JSON.

## 4 Key Learnings from Vapi Integration (Code-Based)

### 4.1 1. API-Based Call Triggering

Calls are initiated using a POST request to:

`https://api.vapi.ai/call/phone`

The payload includes:

- `assistantId`
- `phoneNumberId`
- `customer` (name, number)
- `assistantOverrides`

## 4.2 2. Assistant Overrides

The system customizes the assistant dynamically using:

- `firstMessage` (initial greeting)
- `variableValues` (customer name, loan amount, etc.)

## 4.3 3. Webhook Integration

The webhook URL is passed as:

```
serverUrl = PUBLIC_BASE_URL + /api/vapi/webhook
```

This enables Vapi to send transcript and call data back to the backend after call completion.

## 4.4 4. Artifact Configuration

From the code, the following artifacts are enabled:

- `recordingEnabled = True`
- `loggingEnabled = True`
- `transcriptPlan.enabled = True`

This shows that transcripts and recordings are explicitly configured in the request payload.

# 5 NLU Design

The project implements a rule-based Natural Language Understanding (NLU) pipeline consisting of three main components:

- **Intent Classification:** Determines user intent such as interest, denial, or callback request using keyword-based rules.
- **Slot Extraction:** Extracts structured values like monthly income, loan amount, employment type, etc.
- **Analyze Function:** Combines intent and slots into a unified structured output.

The NLU operates on the transcript received after the call and converts unstructured text into usable data.

# 6 Challenges Faced

- Handling multiple numerical values (e.g., income and loan amount) in a single transcript.
- Ambiguity caused by processing the entire transcript as one input.
- Understanding webhook flow and asynchronous communication.
- Ensuring sensitive data is not stored in logs or JSON files.

# 7 Future Improvements

- Process conversation turn-by-turn instead of using full transcript.
- Improve slot extraction to handle multiple values correctly.
- Add decision logic based on extracted financial data.
- Enhance NLU with more robust parsing methods.

## 8 Conclusion

This project demonstrates integration of a voice AI system with a backend using API calls and webhooks. It highlights how structured data can be extracted from conversational transcripts and used for further processing.